



Guide de test sandbox PayTabs & CMI Maroc

Configuration, credentials et scénarios de bout en bout

VERSION 1.0 · MAI 2026 · ROADMAP P1 PAIEMENTS INTERNATIONAUX

Table des matières

01.	Vue d'ensemble & prérequis	3
02.	Setup ngrok (tunnel local)	4
03.	PayTabs — Sandbox immédiat	5
04.	CMI Maroc — Test partiel en local	8
05.	Configuration Clenzy	10
06.	Scénarios de test end-to-end	11
07.	Débogage & commandes utiles	13
08.	Annexe — Script compute-cmi-hash.sh	14

1. Vue d'ensemble & prérequis

Ce guide accompagne le déploiement local des deux nouveaux fournisseurs de paiement intégrés dans Clenzy (roadmap P1) : **PayTabs** pour l'Arabie Saoudite et **CMI** pour le Maroc. L'objectif est de valider l'intégration complète en environnement de test avant le passage en production.

État de la roadmap P1

PR	DESCRIPTION	STATUT	COUVERTURE TESTS
PR1	Correctifs orchestrateur (refund agnostique, transaction-status, CMI webhook body)	Livré	3482 tests verts
PR2	Implémentation PayTabs : client HTTP, provider, webhook HMAC-SHA256	Livré	16 tests dédiés
PR3	Frontend : dialog de configuration des credentials	Livré	TypeScript clean
PR4	Implémentation CMI : hash SHA-512 ver3, redirect controller, webhook	Livré	40 tests dédiés
PR5	Routing currency-aware (SAR → PayTabs, MAD → CMI)	Livré	10 tests dédiés

Prérequis machine

OUTIL	VERSION	VÉRIFICATION
<code>docker</code> + <code>docker compose</code>	recent	<code>docker compose version</code>
Java JDK	21+	<code>java -version</code>
Node.js	20+	<code>node --version</code>
<code>openssl</code> + <code>base64</code>	natif macOS	<code>openssl version</code>
<code>ngrok</code>	3.x	<code>ngrok version</code> — installation détaillée section 2

Ce que ce guide couvre

- Création des comptes sandbox PayTabs et stratégie CMI

- Configuration du tunnel ngrok pour exposer les webhooks locaux
- Saisie des credentials dans Clenzy via l'interface admin
- Tests end-to-end avec cartes bancaires de test
- Simulation manuelle des callbacks CMI (en attendant l'accès sandbox officiel)
- Outils de debug : logs, requêtes SQL, dashboard ngrok

2. Setup ngrok (tunnel local)

Pourquoi ngrok ?

PayTabs et CMI doivent pouvoir **joindre votre serveur Clenzy depuis Internet** pour envoyer les notifications de paiement (IPN). Votre `localhost` n'est pas accessible publiquement. ngrok crée un tunnel HTTPS sécurisé qui relaie les requêtes externes vers votre machine.

Installation

```
$ brew install ngrok/ngrok/ngrok
```

Authentification

- 1 Créer un compte gratuit sur `ngrok.com`
- 2 Copier le token depuis le dashboard ngrok (section *Your Authtoken*)
- 3 Configurer le token localement :

```
$ ngrok config add-authtoken <votre-token-ngrok>
```

Lancer le tunnel sur le port backend Clenzy

```
$ ngrok http 8084
```

Une fois lancé, ngrok affiche une URL publique du type :

```
Forwarding https://a1b2c3d4.ngrok-free.app -> http://localhost:8084
```

Notez cette URL : c'est elle que vous allez configurer dans PayTabs et dans Clenzy comme **Callback URL**.

URL ngrok éphémère

Sur le plan gratuit ngrok, l'URL change à chaque relance. Si vous redémarrez ngrok, mettez à jour l'URL dans le dashboard PayTabs (section IPN) **et** dans la configuration Clenzy (champ Callback URL du dialog provider). Le plan payant ngrok permet d'avoir un sous-domaine fixe.

Vérification

```
$ curl https://a1b2c3d4.ngrok-free.app/api/webhooks/payments/paytabs
```

Doit retourner `401 Missing signature` — c'est attendu, ça confirme que le tunnel fonctionne et que la route est bien exposée.

Dashboard ngrok

Pendant que ngrok tourne, ouvrir `http://127.0.0.1:4040` dans un navigateur pour voir **en temps réel toutes les requêtes entrantes** avec leurs headers, body et réponses. Indispensable pour debugger les webhooks.

3. PayTabs – Sandbox immédiat DISPONIBLE

PayTabs offre un sandbox **auto-service, gratuit et sans KYB**. Vous pouvez créer un compte de test en 5 minutes et obtenir immédiatement des credentials fonctionnels.

3.1 – Création du compte marchand sandbox

- 1 Ouvrir <https://merchant.sandbox.paytabs.com/auth/register>
- 2 Sélectionner le pays **Saudi Arabia** (région principale ciblée par Clenzy)
- 3 Renseigner email, mot de passe, nom d'entreprise (n'importe lequel, aucune vérification)
- 4 Confirmer l'email reçu dans la boîte de réception
- 5 Se connecter au dashboard : `merchant.sandbox.paytabs.com`

3.2 – Récupération des credentials

CREDENTIAL	OÙ LE TROUVER	CHAMP CLENZY
Server Key (secret)	Menu <code>Developers</code> → <code>Server Key</code> → bouton "Show"	<code>apiKey</code> (Server Key dans le dialog)
Profile ID (numérique)	Page <code>Profile</code> → en haut, sous le nom du marchand	<code>configJson.profileId</code>
Region	Profile → "Region" (sera <code>SA</code>)	<code>configJson.region</code>

Format attendu du Server Key : `STJN6XX12-XXXXX-XXXXX-XXXXX`

3.3 – Activation de l'IPN (webhook)

L'IPN n'est PAS activé par défaut

Sans cette étape, PayTabs effectuera le paiement mais ne notifiera jamais Clenzy → la transaction restera en `PROCESSING` indéfiniment.

- 1 Dashboard PayTabs → menu `Developers` → onglet `IPN`
- 2 Activer l'IPN (toggle)

3

Renseigner l'URL :

`https://<votre-tunnel-ngrok>/api/webhooks/payments/paytabs`

4

Cocher les événements :

- `Sale Successful`
- `Sale Declined`
- `Sale Error`

5

Sauvegarder

3.4 — Cartes de test PayTabs

Cartes officielles fournies par PayTabs pour simuler des scénarios :

SCÉNARIO	NUMÉRO DE CARTE	EXP.	CVV	OTP 3DS
Succès complet	<code>4111 1111 1111 1111</code>	12/30	123	<code>1234</code>
Carte déclinée	<code>4012 0010 3714 1112</code>	12/30	123	—
Échec 3D Secure	<code>4000 0000 0000 0002</code>	12/30	123	<code>0000</code>
Erreur générique	<code>4012 0010 3844 0026</code>	12/30	123	—

Le **nom du porteur** et l'**adresse** peuvent être n'importe quelle valeur en sandbox — aucune vérification.

Documentation officielle des cartes de test : docs.paytabs.com/manuals/PT-Sandbox-and-Cards/

4. CMI Maroc — Test partiel en local SANDBOX NDA

Pas de sandbox public

Contrairement à PayTabs, CMI Maroc **ne propose pas de sandbox auto-service**. L'obtention de credentials de test requiert :

- Une entité légale au Maroc (SARL, SA ou auto-entrepreneur marocain)
- Signature d'un NDA
- Délai de 2 à 4 semaines
- Documents : KBis marocain (RC), RIB d'une banque marocaine, CIN du gérant

Contact commercial CMI : `commercial@cmi.co.ma` · téléphone `+212 522 366 100`

4.1 — Approche court terme : tester le flow Clenzy avec credentials fictifs

En attendant l'accès officiel, on peut **valider la quasi-totalité du flow Clenzy** avec des credentials fictifs :

- Génération correcte du HASH SHA-512 ver3 ✓
- Construction des paramètres CMI (clientid, amount, currency, oid...) ✓
- Rendu du HTML auto-submit par le `CmiRedirectController` ✓
- Vérification de la signature dans le webhook handler ✓
- Routage `orchestrationService.completeTransaction()` ✓

La seule chose qu'on ne peut *pas* valider sans accès officiel : la connexion réseau effective au portail `testpayment.cmi.co.ma` (qui rejettera évidemment le `clientId` inconnu).

4.2 — Credentials fictifs à utiliser dans Clenzy

CHAMP	VALEUR DE TEST
Client ID	<code>TEST_CLIENT_001</code>
Store Key	<code>STORE_KEY_LOREM_IPSUM_DOLOR_SIT_AMET</code>
OK URL	<code>http://localhost:3000/booking/success</code>
Fail URL	<code>http://localhost:3000/booking/error</code>
Callback URL	<code>https://<tunnel-ngrok>/api/webhooks/payments/cmi</code>
Mode sandbox	Activé (toggle ON)

4.3 — Simulation du callback IPN avec le script `compute-cmi-hash.sh`

Pour valider que la vérification du HASH SHA-512 ver3 fonctionne **sans attendre CMI**, on peut générer manuellement un HASH valide et envoyer un faux callback :

```
$ ./compute-cmi-hash.sh "STORE_KEY_LOREM_IPSUM_DOLOR_SIT_AMET" \  
"clientid=TEST_CLIENT_001" \  
"oid=TX-abc12345" \  
"amount=250.00" \  
"currency=504" \  
"ProcReturnCode=00" \  
"Response=Approved"
```

Le script produit :

1. Le HASH calculé (Base64, 88 caractères)
2. Le plaintext utilisé (pour debug)
3. Une commande `curl` prête à exécuter

Important : créer d'abord la transaction

Le webhook handler vérifie que l' `oid` correspond à une `PaymentTransaction` existante. Avant d'envoyer le faux callback, lancez d'abord une vraie transaction CMI depuis le frontend Clenzy (sélection d'une propriété marocaine, devise MAD) — cela crée la `PaymentTransaction` en BDD avec un `transactionRef` unique. Reprenez ce `transactionRef` comme valeur d' `oid` dans votre commande `curl`.

4.4 — Validation attendue côté Clenzy

SCÉNARIO	RÉPONSE HTTP	EFFET BDD
HASH valide + <code>ProcReturnCode=00</code>	<code>200 OK</code>	Transaction → <code>COMPLETED</code> , event Kafka publié
HASH invalide	<code>401 Invalid HASH</code>	Aucun changement
<code>oid</code> inconnu	<code>404 Unknown transaction</code>	Aucun changement
<code>ProcReturnCode</code> ≠ "00"	<code>200 OK</code>	Transaction → <code>FAILED</code> avec le message d'erreur

5. Configuration Clenzy

5.1 — Variables d'environnement

Dans `server/src/main/resources/application.yml` ou `.env.local`, vérifier ces variables :

VARIABLE	VALEUR ATTENDUE (DEV)	RÔLE
<code>clenzy.base-url</code>	<code>http://localhost:8084</code>	Utilisé par CMI pour construire le redirectUrl
<code>SPRING_PROFILES_ACTIVE</code>	<code>dev</code>	Active la config de sécurité permissive

Override possible en variable d'env :

```
$ export CLENZY_BASE_URL=http://localhost:8084
```

5.2 — Saisie des credentials via l'interface admin

- 1 Login Clenzy en tant que `SUPER_ADMIN`
- 2 Aller dans **Paramètres** → onglet **Paiement**
- 3 Identifier le provider à configurer (PayTabs ou CMI) — chip "**À configurer**" en orange
- 4 Cliquer sur l'icône  (Configurer) à droite de la ligne
- 5 Renseigner les champs du dialog (cf. sections 3.2 ou 4.2)
- 6 Vérifier que **Mode sandbox** est activé
- 7 Cliquer sur **Enregistrer**
- 8 Le toggle du provider s'active automatiquement

5.3 — Sécurité des credentials

Chiffrement automatique côté backend

Tous les secrets (`Server Key` , `Store Key` , `Webhook Secret`) sont automatiquement chiffrés en **AES-256-GCM** via Jasypt avant d'être stockés en base. L'API REST ne les retourne **jamais** en clair — si vous rouvrez le dialog de config, le champ secret apparaît vide (laisser vide = préserver la valeur existante).

6. Scénarios de test end-to-end

6.1 — Scénario complet PayTabs (devise SAR)

Préparation : backend Clenzy démarré sur `:8084`, frontend sur `:3000`, ngrok actif, compte sandbox PayTabs créé et configuré.

- 1 Booking Engine → sélectionner une propriété en KSA (devise SAR)
- 2 Compléter la réservation → page paiement
- 3 Frontend appelle `POST /api/payments/create-session` avec `currency: "SAR"`
- 4 `PaymentOrchestrationService.resolveProvider()` → identifie SAR → cherche PayTabs enabled → trouve → utilise `PayTabsPaymentProvider`
- 5 `PayTabsPaymentProvider.createPayment()` → POST vers `secure.paytabs.sa/payment/request` → reçoit `redirect_url`
- 6 Frontend redirige le navigateur vers le portail PayTabs
- 7 Saisir la carte `4111 1111 1111 1111`, exp `12/30`, CVV `123`
- 8 Page 3D Secure → OTP `1234`
- 9 Redirection vers `successUrl` (Clenzy)
- 10 **En parallèle** : PayTabs poste un IPN sur `https://<ngrok>/api/webhooks/payments/paytabs`
- 11 `PaymentWebhookRouter.handlePayTabsWebhook()` : parse `cart_id` → charge le Server Key → vérifie HMAC-SHA256 → si `response_status == "A"` → `completeTransaction()`
- 12 BDD : transaction → `COMPLETED`, réservation → `PAID`

6.2 — Scénario partiel CMI (devise MAD, sans sandbox CMI)

- 1 Configurer CMI avec les credentials fictifs de la section 4.2
- 2 Forcer une transaction CMI (depuis le frontend en MAD, ou via curl) :

```
$ curl -X POST http://localhost:8084/api/payments/create-session \
-H "Authorization: Bearer <jwt-keycloak>" \
-H "Content-Type: application/json" \
-d '{"amount":250.00,"currency":"MAD","sourceType":"RESERVATION","sourceId":1}'
```

- 3 Le backend retourne `redirectUrl: "http://localhost:8084/api/payments/cmi-redirect/TX-xxx"`
- 4 Ouvrir cette URL dans le navigateur → page de redirection Clenzy (spinner + "Redirection vers CMI...")
- 5 Le formulaire HTML auto-submit vers `testpayment.cmi.co.ma`
- 6 CMI rejette (clientid inconnu) → c'est attendu, le flow Clenzy est validé jusqu'à la redirection
- 7 **View source** de la page pour vérifier :
 - Tous les paramètres CMI présents (clientid, amount, currency=504, oid, HASH...)
 - Le HASH ressemble à du Base64 (88 caractères, terminant éventuellement par `==`)
 - L'URL d'action est `https://testpayment.cmi.co.ma/fim/est3Dgate`
- 8 Simuler le callback avec le script `compute-cmi-hash.sh` + curl (section 4.3)
- 9 Vérifier que la transaction passe en `COMPLETED`

6.3 — Tests de fallback du routing

Le `PaymentOrchestrationService.resolveProvider()` applique cette logique de priorité :

1. **preferredProvider explicite** : si l'appelant impose un provider, on l'utilise
2. **Currency match** : SAR → PayTabs (si enabled pour l'org), MAD → CMI (si enabled)
3. **Country match** : premier provider enabled pour le pays de l'org
4. **Fallback Stripe** : dernière chance

Tester en désactivant successivement les providers et en lançant des transactions avec différentes devises :

CONFIG ORG	DEVISE TRANSACTION	PROVIDER ATTENDU
Stripe ON, PayTabs ON	SAR	PayTabs (currency match)
Stripe ON, PayTabs OFF	SAR	Stripe (fallback)
Stripe ON, CMI ON	MAD	CMI (currency match)
Stripe ON, CMI ON	EUR	Stripe (pas de currency match fort sur EUR)
Stripe OFF, PayTabs ON	EUR	PayTabs (seul enabled, mais non-natif EUR)

7. Débogage & commandes utiles

7.1 — Logs backend Clenzy

```
$ tail -f server/logs/application.log | grep -i "paytabs\|cmi\|payment"
```

Logs clés à surveiller :

- `PayTabs payment created – tranRef=XXX for org=YY` : provider créé avec succès
- `PayTabs webhook : transaction TX-xxx confirmée` : IPN reçu et traité
- `CMI redirect rendu pour txRef=XXX → https://testpayment.cmi.co.ma/...` : redirection servie
- `CMI webhook : HASH invalide pour oid=...` : problème de signature (vérifier le store_key)

7.2 — Inspection BDD

```
$ docker exec -it clenzy-postgres-dev psql -U clenzy -d clenzy -c \  
"SELECT transaction_ref, provider_type, status, amount, currency,  
       created_at, error_message  
FROM payment_transactions  
ORDER BY created_at DESC LIMIT 10;"
```

```
$ docker exec -it clenzy-postgres-dev psql -U clenzy -d clenzy -c \  
"SELECT provider_type, enabled, country_codes, sandbox_mode, config_json  
FROM payment_method_configs  
WHERE organization_id = 1;"
```

7.3 — Dashboard ngrok

Pendant que ngrok tourne, ouvrir `http://127.0.0.1:4040` :

- Onglet **Inspect** : voir chaque requête entrante en détail (body, headers, status, durée)
- Onglet **Status** : statistiques de trafic du tunnel
- Bouton **Replay** : rejouer une requête (utile pour tester un IPN sans repasser par le portail)

7.4 — Tests unitaires backend

```
$ cd server && mvn test -  
Dtest='PayTabsPaymentProviderTest,CmiHashServiceTest,CmiPaymentProviderTest'
```

7.5 — Erreurs courantes

SYMPTÔME	CAUSE PROBABLE	SOLUTION
PayTabs renvoie HTTP 401 sur createPayment	Server Key incorrect ou non lu depuis la BDD	Vérifier que le toggle PayTabs est activé en BDD ET que <code>apiKey</code> n'est pas null
Transaction reste en PROCESSING	IPN PayTabs non activé dans le dashboard sandbox	Cf. section 3.3 — activer l'IPN + vérifier l'URL ngrok
CMI webhook renvoie 401 "Invalid HASH"	Le store_key utilisé pour calculer le HASH dans le script ne matche pas celui configuré dans Clenzy	Vérifier que les deux valeurs sont strictement identiques (sensible à la casse)
CMI redirect : "Transaction introuvable"	L'URL <code>/cmi-redirect/<txRef></code> contient un txRef inexistant en BDD	Lancer d'abord une vraie transaction depuis le frontend
ngrok change d'URL à chaque relance	Plan gratuit ngrok	Mettre à jour l'URL dans PayTabs ET dans Clenzy à chaque relance, ou passer au plan payant

8. Annexe — Script `compute-cmi-hash.sh`

Le script `compute-cmi-hash.sh` est livré dans `docs/payments-sandbox/`. Il calcule le HASH SHA-512 ver3 selon la spec officielle CMI, en parité stricte avec l'implémentation Java `CmiHashService` du backend Glenzy.

Algorithme reproduit

1. Trier les paramètres par nom de champ (case-insensitive, ordre alphabétique)
2. Exclure les champs `HASH` et `encoding` du calcul
3. Échapper chaque valeur : `\` → `\\`, puis `|` → `\\|`
4. Joindre les valeurs échappées avec `|`
5. Concaténer `|` + `escape(storeKey)` à la fin
6. Calculer SHA-512 sur les bytes UTF-8 → encoder en Base64 standard

Dépendances

Uniquement des outils Unix natifs : `openssl`, `base64`, `sort`, `tr`. Aucune installation requise sur macOS et Linux.

Validation

Le script a été validé en parité contre :

- L'implémentation Java `CmiHashService` (26 tests unitaires verts)
- Une implémentation de référence en Python (algorithme officiel)
- Cas d'échappement spéciaux : `|`, `\`, et combinaisons

Usage

```
$ ./compute-cmi-hash.sh STORE_KEY "key1=value1" "key2=value2" ...

# Options
--webhook-url URL      URL du webhook pour la commande curl générée
                        Default: http://localhost:8084/api/webhooks/payments/cmi
-h, --help            Affiche l'aide
```

Exemple d'utilisation complète

```
$ ./compute-cmi-hash.sh \  
"STORE_KEY_LOREM_IPSUM_DOLOR_SIT_AMET" \  
"clientid=TEST_CLIENT_001" \  
"oid=TX-abc12345" \  
"amount=250.00" \  
"currency=504" \  
"ProcReturnCode=00" \  
"Response=Approved"
```

Le script produit en sortie :

1. Le HASH calculé (Base64, 88 caractères)
2. Le plaintext utilisé pour le hash (debug)
3. Une commande `curl` prête à exécuter avec tous les paramètres + le HASH
4. Une description des réponses HTTP attendues côté Clenzy

Vous voilà prêt

Avec ce script + le sandbox PayTabs gratuit + ngrok, vous disposez de tout ce qu'il faut pour tester les deux providers sans attendre l'accès officiel à CMI. Pour le passage en production, il restera à obtenir les credentials prod auprès de chaque marchand et à les saisir via le même dialog de configuration — le code Clenzy est déjà prêt.